GNU MAKE

Build Commands, Flags & Workflows

Beginner to advanced • Embedded firmware, C/C++, Linux & RTOS

Purpose: A practical guide to the make command, common flags, build targets, clean/rebuild workflows, troubleshooting, and safe habits for embedded software engineers.

Important: GNU Make is a build tool. Commands such as make, make clean, and make linux_all behave according to the Makefile(s) in your project. Target names such as linux_all, flash, rebuild, package, or test are project-defined unless documented otherwise.

Coverage note: This guide covers common GNU Make commands and flags, not every possible flag in every Make version or every custom build system. Always check make --help and the project's Makefile.

1. What is Make?

Make reads build rules from a Makefile and decides which targets need to be updated. In a C/C++ project, a build may compile source files into object files and link them into an executable, library, or firmware image.

Term	Simple meaning	Embedded example
make	The build tool command.	Run the project's default build target.
Makefile	File containing rules, dependencies, variables, and recipes.	Defines how drivers and applications are compiled.
Target	A named result or action described in the Makefile.	all, clean, linux_all, firmware, test.
Prerequisite / dependency	A file or target that another target needs first.	An object file depends on its C file and header dependencies.
Recipe	Shell command(s) Make runs to build a target.	A compiler or linker invocation.
Compiler	Translates source code into object code.	arm-none-eabi-gcc or another toolchain compiler.
Linker	Combines object files and libraries into an output.	Creates an ELF firmware image.
Artifact	A generated output from a build.	.elf, .hex, .bin, map file, library, package.

2. The most important rule: target names are not universal

make clean is common, but it only does what the clean target in your project says. make linux_all is not a built-in GNU Make command; it asks Make to build a target named linux_all. One project might build Linux applications, another might assemble a complete product image, and another might not have that target at all.

```
make --help
make -qp 2>/dev/null | less
# Inspect project files before choosing a target
find . -type f \( -name 'Makefile' -o -name 'makefile' -o -name 'GNUmakefile' -o -name '*.mk' \)
```

Use the project's README/build guide first. Do not assume a target's meaning from its name alone.

3. Beginner daily commands

Command	What it does	When / impact
make	Builds the default goal (often the first ordinary target).	Use for a normal build if project documentation says so. It may not build every component.
make all	Runs target all, if defined.	Often means build the main set of outputs; project-defined.
make clean	Runs clean target, if defined.	Often deletes object files and generated outputs. Inspect recipe before using.
make linux_all	Runs target linux_all, if defined.	Use only when project documentation says it builds the desired Linux/product configuration.
make -j4	Allows up to four jobs to run in parallel.	Often faster; may use more RAM and expose broken dependency rules.
make -j$(nproc)	Uses number of online processing units on many Linux systems.	Convenient on Linux shells; high parallelism can exhaust memory.
make help	Runs help target if defined.	Shows project targets only if maintainers implemented a help target.
make -n all	Prints commands Make would run for all without executing recipes.	Useful to preview a target; Make may still read/expand Makefile functions.
make -C path/to/project	Changes to a directory before	Useful in scripts or from a different

	reading/running Make.	working directory.
make -f Custom.mk	Uses a specified Makefile.	Useful when a repository has multiple build definitions.

4. Clean, rebuild, and forced build

4.1 make clean

```
make clean
```

Why: removes files or directories listed in the project's clean recipe. When: stale object files, switching configuration/toolchain, or following project build instructions. Impact: generated files may be deleted; source files should normally remain, but the recipe is authoritative.

4.2 Clean then build

```
make clean
make -j4 linux_all
```

Why: ensures the selected target is built after the clean step. When: you need a fresh build after configuration or toolchain changes. Impact: costs more time; confirm linux_all exists and that clean is safe.

4.3 make clean all is not the same as a guaranteed sequential rebuild

```
make clean all
```

This asks Make to update both targets in one invocation; it is not a universal guarantee that clean finishes before all starts in every dependency arrangement. Avoid combining clean and build targets with parallel jobs. Two separate invocations are clearer and safer.

4.4 make -B

```
make -B all
```

Why: tells Make to consider targets out of date and rebuild them. When: intentionally forcing a rebuild for diagnosis. Impact: can compile much more than needed and take longer; it does not clean arbitrary generated files.

4.5 distclean / cleanall / rebuild

These names are conventions, not built-in targets. distclean often removes configuration outputs as well as build outputs; cleanall and rebuild vary by project. Check their recipes before running them, especially in vendor SDKs or large product trees.

5. Common flags and options

Flag / option	Meaning	When to use / impact
-jN	Run up to N jobs concurrently; e.g. -j4.	Speeds up builds, uses more CPU/RAM. A bare -j may allow very high parallelism.
-n	Dry run: print recipe commands without executing them in ordinary cases.	Preview an unfamiliar build. It is not a complete security sandbox; Makefile parsing and expansion still occur.
-B	Consider targets out of date; force rebuilding.	Useful for diagnosis, but can make builds slower.
-k	Continue with other independent targets after an error.	Useful to collect multiple errors; output may contain many secondary failures.
-s	Silent mode: do not echo recipe commands.	Less output, but harder to debug.
-w	Print entering/leaving directory messages.	Useful for recursive or multi-directory builds.
-C DIR	Change to DIR before reading	Run a build from another directory.

	Makefiles.	
-f FILE	Read FILE as the Makefile.	Select a custom build file.
-p	Print Make's database of variables and rules.	Advanced diagnosis; output can be very large.
-q	Question mode: report whether targets are up to date via exit status.	Useful in scripts; does not build the target.
-r	Disable built-in implicit rules.	Advanced troubleshooting; may break builds relying on built-in rules.
-R	Disable built-in variables as well as built-in rules.	Advanced; can alter expected tool/flag defaults.
-t	Touch targets rather than running normal recipes in common cases.	Special use only; can make timestamps misleading if misunderstood.
-o FILE	Treat FILE as old for dependency decisions.	Advanced, rarely needed in daily firmware work.
-W FILE	Pretend FILE was just modified for dependency decisions.	Useful to test rebuild logic.
--warn-undefined-variables	Warn about undefined variables in Makefile expansion.	Useful for diagnosing misspelled variables; can generate noise.
--no-print-directory	Suppress directory messages when enabled by recursive Make.	Cleaner logs.

Flags can be combined, for example make -j4 -k. Short options are not interchangeable: the meaning of each flag depends on Make itself, not on Git.

6. Verbose builds and compiler flags

6.1 V=1 is a project convention

```
make V=1
```

Many Linux kernel and embedded build systems use V=1 to print full compiler/linker commands. GNU Make itself does not give V=1 a special universal meaning; the Makefile must use the variable.

6.2 Common compiler/build variables

Variable / option	Typical purpose	Caution
CC	C compiler command, often gcc or a cross-compiler.	Project may override it.
CXX	C++ compiler command.	Must match intended toolchain.
CFLAGS	C compiler flags in many Makefiles.	Exact behavior depends on Makefile.
CPPFLAGS	Preprocessor/include flags by convention.	Often carries -I include paths and -D definitions.
LDFLAGS	Linker flags by convention.	A wrong linker script or memory setting can break firmware.
LDLIBS	Libraries passed during linking by convention.	Order can matter for some linkers.
CROSS_COMPILE	Cross-toolchain prefix used by some projects, especially Linux kernel builds.	Not universal; use the project's required syntax.
ARCH	Target architecture variable in some build systems.	Project-specific; not a general GNU Make variable.
O=DIR	Out-of-tree output directory in some systems, notably Linux kernel build workflows.	Not universal; check project docs.

6.3 Example: inspect before overriding variables

```
make -n CC=arm-none-eabi-gcc all
make V=1 all
```

These examples are illustrative. Overriding CC may be insufficient if the project uses a toolchain file, wrapper, compiler prefix, or generated configuration. Do not change production build flags without knowing the expected toolchain.

7. Find available targets and understand the Makefile

7.1 Check the project help

```
make help
```

If there is no help target, Make may report 'No rule to make target help'. That does not mean Make is broken; the project may not define help.

7.2 Inspect target definitions

```
grep -nE '^[[:alnum:]_.-]+[[:space:]]*:' Makefile
find . -type f \( -name 'Makefile' -o -name '*.mk' \)
```

This is a quick search, not a perfect target inventory. Targets can be included from other files, generated dynamically, use pattern rules, or be defined conditionally.

7.3 Ask Make to print its database

```
make -qp 2>/dev/null | less
```

This advanced method can reveal rules and variables without building in many cases, but it can still evaluate Makefile functions and read included files. Inspect unfamiliar Makefiles before running them.

7.4 Common Makefile concepts

Concept	Simple meaning
all: app firmware	Target all depends on app and firmware targets.
clean:	Target used to remove selected generated files.
.PHONY: all clean	Declares action targets that are not files, so same-named files do not prevent execution.
VAR = value	Recursively expanded variable assignment.
VAR := value	Simply expanded variable assignment; evaluated at assignment time.
VAR ?= value	Set variable only if it is not already set.
VAR += value	Append to a variable value.
$(VAR)	Expand a Make variable.
$@	Automatic variable for the current target.
$<	First prerequisite in common rule forms.
$^	All prerequisites, often listed once each.
%	Pattern character used in pattern rules.
include other.mk	Read additional Makefile content.

8. Build system vs Make vs shell

- Make reads dependency rules and decides which recipes need to run.
- The shell executes recipe commands such as gcc, ld, cp, rm, scripts, or vendor tools.
- The compiler turns C/C++ into object code; the linker combines objects and libraries.
- CMake, Meson, or another build system may generate Makefiles. In that case, you may need to configure the build directory before running Make.
- A Make target named flash, package, deploy, or release can perform significant actions; inspect its recipe before use.

9. Embedded firmware build workflow

1. Read README/build instructions and identify the correct target board, configuration, toolchain, and SDK/RTOS version.
2. Check the working tree and branch before making changes: git status.
3. Find the documented build target; do not guess that linux_all means the same thing in every repository.
4. Run the normal incremental build first if the configuration has not changed.
5. If instructed or if stale generated outputs are suspected, run the documented clean target and rebuild separately.
6. Capture the first meaningful compiler or linker error and identify the source file, target, toolchain, and configuration involved.
7. Run required tests and, where applicable, bench or hardware-in-the-loop validation.
8. Record the source commit, toolchain/configuration, build result, and artifact identity for traceability.

10. What common targets might mean

Target name	Common convention only	Verify before use
all	Build a main collection of outputs.	What exact targets are prerequisites?
clean	Remove intermediate build outputs.	Which paths are removed?
distclean	Remove more generated/configuration files.	Could local configuration be deleted?
linux_all	Project-specific Linux build.	Kernel, applications, rootfs, or whole product?
firmware	Build firmware outputs.	Which board/configuration and file formats?
test / check	Run tests or checks.	Does it run host tests, target tests, or both?
flash / program	Program a device.	Could it erase or overwrite a connected target?
package / release	Create distribution/release output.	Does it sign, publish, or upload anything?
install	Copy outputs to a location.	What destination and permissions are used?
menuconfig / defconfig	Common in configurable systems such as Linux kernel builds.	Is the target provided by this project and what config will change?

11. Troubleshooting

Symptom	What to check
No rule to make target	Target may not exist, spelling may be wrong, wrong directory or Makefile may be selected.
Nothing to be done	Make thinks the target is up to date; check prerequisites, timestamps, and whether the target is phony.
Compiler not found	Check toolchain installation, PATH, compiler variable, cross-compiler prefix, and environment setup.
Header not found	Check include paths, generated headers, configuration, and dependency setup.
Undefined reference	Check missing object/library, link order, conditional compilation, and whether implementation was built.
Works locally but fails in CI	Compare toolchain versions, environment variables, dependencies, paths, case sensitivity, and build flags.
Stale output after changing config	Follow the project's clean/reconfigure instructions; some build systems need generated dependency/configuration updates.
Out of memory with -j	Reduce parallel jobs, e.g. make -j2.
Many errors after one failure	Use the first meaningful error; -k may report additional

	independent failures.
Wrong output architecture	Verify target configuration, compiler, architecture variables, and build directory.

12. Safety and impact checklist

- Inspect clean, distclean, flash, install, package, and release recipes before running unfamiliar targets.
- Do not use sudo with make unless official project instructions explicitly require it and you understand why.
- Avoid make clean all combined with parallel execution; use separate commands for predictable ordering.
- Do not assume -B deletes outputs; it forces Make's rebuild decisions.
- Do not assume -n makes an untrusted Makefile completely safe; Makefile evaluation can have side effects.
- Use the same documented compiler, SDK, RTOS, configuration, and flags as the team build.
- A successful compile does not prove firmware works on real hardware.
- Keep production credentials and signing keys out of ordinary build logs and source control.

13. Example command sets

Normal incremental build

```
make help
make -j4 all
```

Use only if all is a documented target. It normally rebuilds outputs Make considers out of date.

Clean rebuild of a project target

```
make clean
make -j4 linux_all
```

Use only after verifying both targets and the clean recipe. Replace linux_all with the documented target for your board/product.

Debug why commands run

```
make -n linux_all
make V=1 linux_all
```

The first previews commands; the second requests verbose output only if the Makefile supports V=1.

Lower parallelism after memory failure

```
make -j2 all
```

Reducing parallel jobs can lower memory pressure. It does not fix a compiler error or incorrect dependency.

Use a separate Makefile

```
make -f Makefile.debug all
```

Use only if the repository provides that file and documents its purpose.

14. Quick revision

Question	Short answer
Is make a compiler?	No. Make orchestrates commands using build rules; compilers and linkers do the actual translation/linking.
Is make clean built in?	No. It is a conventional target that the project must define.
Is make linux_all universal?	No. It is project-defined.
What does -j do?	Allows parallel jobs; faster but uses more resources.
What does -n do?	Prints commands Make would run in ordinary cases, without

	executing recipes.
What does -B do?	Forces targets to be considered out of date; it does not clean arbitrary files.
What does V=1 do?	A common project convention for verbose output; not a universal Make flag.
Why does Make say nothing to do?	It considers the target up to date based on its rules and timestamps.
Why clean before rebuild?	To remove selected generated outputs and reduce stale-build risk; it costs time.
Does a successful build prove hardware works?	No. Target hardware tests may still be required.

15. Official references

Use the version of GNU Make and the build system adopted by your project. Project Makefiles and build guides define the real targets and variables.

- **GNU Make manual:** https://www.gnu.org/software/make/manual/
- **GNU Make options summary:** https://www.gnu.org/software/make/manual/html_node/Options-Summary.html
- **GNU Make targets and prerequisites:** https://www.gnu.org/software/make/manual/html_node/Rule-Introduction.html
- **Linux kernel build system documentation:** https://docs.kernel.org/kbuild/

Final recommendation: learn the common commands here, but use the actual Makefile and project build guide to determine which targets exist, what they build, and what they delete or change.

Compiler Flow: From Source Code to Firmware

This section explains how a C/C++ embedded project turns source files into an executable firmware image. GNU Make normally coordinates the steps; the compiler, assembler, and linker perform the translation work. Exact tools and outputs depend on your project and target board.

1. Compiler/build flow diagram

1. Source files main.c, driver.c, app.cpp
↓
2. Preprocessor Expands #include, #define, and conditional compilation → preprocessed source (.i / .ii)
↓
3. Compiler Checks syntax and types; converts C/C++ into assembly (.s)
↓
4. Assembler Converts assembly into object files (.o / .obj)
↓
5. Linker Combines object files and libraries; resolves symbols → ELF / executable
↓
6. Post-build tools (if configured) Generate .hex / .bin, map files, checksums, signed images, or packaged artifacts
↓
7. Flash / deploy (separate target or tool) Program the board or deliver the image; this does not happen just because compilation succeeded
↓
8. Verify Run tests, inspect size/map, boot the board, and check logs

2. What each stage does

- **Source code:** Developers write .c/.cpp files and headers (.h/.hpp). Headers provide declarations and shared definitions; they are usually included by source files rather than compiled as standalone files.
- **Preprocessing:** The preprocessor handles #include, #define, and #if/#ifdef. A missing header or wrong macro can cause errors or change which code is built.
- **Compilation:** The compiler checks language rules and translates each source file. Warnings can reveal bugs; optimization flags such as -O0 or -O2 can affect speed, size, and debugging.
- **Assembly:** The assembler turns assembly instructions into machine-code object files. Object files are not normally the final flashable image because references to other files may still be unresolved.
- **Linking:** The linker combines object files and libraries, resolves symbols, and applies the linker script/memory layout. Undefined references, duplicate symbols, or memory-region overflow often appear here.
- **Post-build conversion:** Embedded projects may convert ELF to Intel HEX (.hex) or raw binary (.bin), calculate checksums, sign images, or package update bundles. These actions only happen if the build rules configure them.
- **Flashing and validation:** Flashing is a separate operation in many projects. Confirm the correct board, target, image, and connection before programming; then verify startup, logs, and tests.

3. Where GNU Make fits

Make is usually the coordinator, not the compiler itself. It reads a Makefile, checks targets and prerequisites, and runs the recipes needed to produce an output. For example, a target may tell Make to run the cross-compiler, assembler, linker, and conversion tool.

```
make -n                 # print planned commands without running them
make -j4                # build with up to four jobs, if dependencies allow
make clean              # run the project's clean recipe; inspect it first
make linux_all          # only works if this target is defined by your project
make V=1                # commonly enables verbose output in projects that support it
```

Important: `make` does not automatically know what 'firmware', 'clean', 'flash', or 'linux_all' means. The Makefile defines those targets and the commands behind them.

4. Example: one source file through the toolchain

This is an illustrative GCC-style example. Real embedded projects usually add include paths, target CPU flags, startup files, linker scripts, libraries, and a cross-compiler prefix.

```
arm-none-eabi-gcc -E -Iinclude main.c -o main.i
arm-none-eabi-gcc -S -mcpu=cortex-m4 -mthumb main.i -o main.s
arm-none-eabi-gcc -c -mcpu=cortex-m4 -mthumb main.c -o main.o
arm-none-eabi-gcc main.o startup.o -T linker_script.ld -o firmware.elf
arm-none-eabi-objcopy -O ihex firmware.elf firmware.hex
```

The example skips some details for readability. A project's Makefile may combine stages, use different tools, or create different output names. Do not copy CPU flags or linker-script names unless they match your board/project.

5. Common failures and where to look

Message / symptom	Likely stage	What to check
fatal error: xxx.h: No such file	Preprocessing	Include path, filename spelling, dependency/package availability
expected ';' / unknown type	Compilation	Source syntax, declarations, macros, language standard
undefined reference to ...	Linking	Missing object/library, wrong library order, C/C++ name mismatch
region RAM/FLASH overflow	Linking / memory layout	Map file, linker script, section sizes, optimization, feature size
wrong instruction / architecture error	Compile or assemble	Target CPU flags and selected toolchain
ELF builds but board will not boot	Post-build / flash / runtime	Correct image and board, startup code, clock, memory layout, flashing settings, logs

6. Recommended practical workflow

9. Read the README/build instructions and identify the required toolchain and board configuration.
10. Use `make help` if the project provides it; otherwise inspect the Makefile and included build files for target names.
11. Start with a normal build, then use verbose or dry-run mode when diagnosing the commands being executed.
12. Read the first meaningful error in the log; later errors may be consequences of the first failure.
13. After a successful build, check the expected ELF/HEX/BIN and map/size reports if available.
14. Flash only when you know which target/image is being programmed, then run smoke tests and capture logs.
15. Before changing flags, confirm whether they apply to preprocessing, compilation, linking, or post-build steps.

7. Extra topics worth adding to a firmware build guide

- Cross-compilation: host computer versus target CPU; tool prefixes such as arm-none-eabi- and aarch64-linux-gnu-.
- Linker script and memory map: Flash/RAM regions, vector table, sections, stack/heap, and map-file review.
- Debug versus release: optimization, debug symbols, assertions, logging, and reproducibility.
- Dependency tracking: header changes should rebuild affected object files; generated dependencies are often used.
- Build configuration: board/revision, feature macros, toolchain version, SDK/RTOS version, and environment variables.
- CI build: clean checkout, pinned toolchain, automated tests, artifact retention, and build provenance.
- Safety: distinguish `clean` from `flash`/`erase`; review destructive targets before executing them.

Cross-Compilation: Theory, Commands, Examples and Workflow

This section extends the earlier Make/compiler-flow material. It explains cross-compilation in simple English, why embedded projects need it, how to identify the correct toolchain, common commands and flags, and how Make uses the tools. Commands are examples; always use the compiler, CPU flags, SDK and build instructions specified by your project.

1. What is cross-compilation?

Cross-compilation means compiling software on one computer (the host) to run on a different processor or operating system (the target). For example, you may use an x86-64 Windows or Linux development PC to build firmware for an ARM Cortex-M microcontroller. The computer running the compiler is the host; the board or system that will run the result is the target.

Term	Simple meaning	Example
Host	Computer where you build the software	Development PC
Target	CPU/device that will run the software	Cortex-M4 board or ARM Linux device
Toolchain	Compiler plus related build tools	Compiler, assembler, linker, objcopy, debugger
Cross-compiler	Compiler that emits code for a different target	arm-none-eabi-gcc
Sysroot / target libraries	Target headers and libraries used for an OS-based target	ARM Linux sysroot
ABI	Rules for how compiled code calls functions, passes values and represents data	Hard-float vs soft-float ABI

2. Why and when is it used?

- **Different CPU:** The development PC may use x86-64 while the device uses ARM, RISC-V or another architecture.
- **No full development environment on the target:** Small microcontrollers have limited memory and usually cannot compile their own firmware.
- **Faster development:** A PC can build large projects more quickly and provide editors, debuggers and automation.
- **Repeatable releases:** A controlled toolchain and build configuration help a team produce consistent firmware.
- **Linux device images:** Embedded Linux applications are often built on a PC against the target sysroot and target libraries.

3. Common toolchain families

Tool prefix / command	Typical use	Important note
gcc / g++	Build for the machine running the compiler (native build)	Usually host architecture; confirm with compiler configuration.
arm-none-eabi-gcc / g++	Bare-metal ARM microcontrollers, often Cortex-M	Normally no full Linux operating-system runtime; use the vendor/RTOS SDK as required.
aarch64-linux-gnu-gcc / g++	64-bit ARM Linux applications	Requires compatible target libraries/sysroot and ABI.
arm-linux-gnueabihf-gcc / g++	32-bit ARM Linux with a hard-float ABI	Must match the target's ABI and runtime.
riscv64-unknown-elf-gcc	Common bare-metal RISC-V toolchain naming	Exact prefix varies by toolchain distribution.
Vendor SDK compiler	MCUs/SoCs supported by a vendor SDK	Use the exact version/configuration recommended for the SDK.

4. Cross-compilation flow

A. Choose target Board/CPU, OS or bare-metal, ABI, SDK and required features
↓
B. Select toolchain Correct compiler, assembler, linker, libraries and utilities
↓

C. Configure build Make variables, include paths, CPU flags, sysroot and linker script
↓
D. Compile each source C/C++ source → object files (.o) for the target CPU
↓
E. Link Object files + target libraries + linker script/sysroot → ELF executable
↓
F. Post-process Optional HEX/BIN/package/signing/map/size output
↓
G. Deploy and test Flash microcontroller or copy application to target Linux device; validate on target

5. Useful commands: what, why, when and impact

Command	What it does	Why / when	Impact / caution
which arm-none-eabi-gcc	Finds the compiler executable on Linux/macOS shells.	Use when Make says the compiler is not found.	Shows the path if installed and available on PATH; it does not prove the compiler matches your board.
arm-none-eabi-gcc --version	Displays compiler version.	Check before building or comparing developer/CI machines.	Different versions can change diagnostics, code generation or compatibility.
arm-none-eabi-gcc -dumpmachine	Prints the compiler's configured target triplet.	Confirm what target family the compiler is configured for.	Useful clue, but not a full check of all CPU flags or ABI options.
arm-none-eabi-gcc -E -Iinclude main.c -o main.i	Runs preprocessing only.	Debug missing headers or macro/conditional-compilation issues.	Produces preprocessed source; does not create a final firmware image.
arm-none-eabi-gcc -S main.c -o main.s	Compiles to assembly and stops before assembling.	Inspect generated assembly or investigate code-generation issues.	CPU flags and optimization options affect the assembly.
arm-none-eabi-gcc -c main.c -o main.o	Compiles/assembles one source file to an object file.	Build individual translation units; common Makefile rule.	Object file is not usually flashable by itself; include project-specific flags and include paths.
arm-none-eabi-gcc main.o driver.o -o app.elf	Links object files into an executable.	Illustrates linking; real bare-metal builds need startup objects and a linker script.	This simplified command may fail or create an unusable image for a real MCU project.
arm-none-eabi-objcopy -O ihex app.elf app.hex	Converts an ELF file to Intel HEX.	When the board flashing tool expects HEX.	Only works if objcopy supports the output format and the ELF contains the intended sections.
arm-none-eabi-size app.elf	Reports text/data/bss sizes.	Check memory usage during development or CI.	Useful summary; map file and target memory layout provide more detail.
arm-none-eabi-objdump -h app.elf	Lists ELF sections.	Inspect section placement and sizes.	Does not replace reviewing the linker script and map file.
arm-none-eabi-objdump -d app.elf	Disassembles machine code.	Debug low-level code generation or inspect a binary.	Output can be large; use options or filter relevant symbols.
make CROSS_COMPILE=arm-none-eabi-	Passes a variable to Make.	Used by projects whose Makefiles use CROSS_COMPILE to construct tool names.	Only works if the Makefile honors that variable; some projects use different variable names.

make CC=arm-none-eabi-gcc	Overrides the Make variable CC for this invocation.	Use if the Makefile uses CC to select the C compiler.	A C++ project may also need CXX; the Makefile may override command-line settings in special cases.
make V=1	Common project convention to print full build commands.	Use when the build output hides compiler flags.	Not a built-in GNU Make meaning; only works if the project supports V=1.
make -n	Prints recipes without executing them.	Inspect which compiler, flags and paths would be used.	Dry-run output can differ from real execution for recipes that use recursive Make or special shell behavior.
make -j4	Allows up to four parallel jobs.	Speed up independent compilation tasks.	Build rules must declare dependencies correctly; otherwise parallel builds may expose race conditions.
make clean	Runs the project's clean target.	Remove generated build products before rebuilding.	The exact files deleted are project-defined; inspect the recipe before use.

6. Example A: bare-metal Cortex-M firmware

Illustrative only: use the CPU flags, startup files, linker script, libraries and compiler version required by your board/SDK. A real project should normally build through its supplied Makefile or IDE project rather than copying these commands blindly.

```
arm-none-eabi-gcc --version
arm-none-eabi-gcc -mcpu=cortex-m4 -mthumb -ffunction-sections -fdata-sections -c main.c -o main.o
arm-none-eabi-gcc -mcpu=cortex-m4 -mthumb -c driver.c -o driver.o
arm-none-eabi-gcc -mcpu=cortex-m4 -mthumb startup.o main.o driver.o -T linker_script.ld -Wl,--gc-sections -o firmware.elf
arm-none-eabi-objcopy -O ihex firmware.elf firmware.hex
arm-none-eabi-size firmware.elf
```

Why the flags matter: -mcpu and -mthumb select target instruction behavior; -ffunction-sections/-fdata-sections can place functions/data in separate sections; -T selects the linker script; -Wl, passes an option to the linker. Exact flags vary by MCU and toolchain.

7. Example B: cross-compile an application for embedded Linux

Embedded Linux builds differ from bare-metal firmware. They usually need target Linux headers, libraries, and a compatible sysroot. A compiler that targets AArch64 is not sufficient if the target libraries or ABI do not match.

```
aarch64-linux-gnu-gcc --version
aarch64-linux-gnu-gcc --sysroot=/path/to/sysroot -I/path/to/headers -c app.c -o app.o
aarch64-linux-gnu-gcc --sysroot=/path/to/sysroot app.o -L/path/to/libs -lexample -o app
```

Replace the example paths and library name with those provided by your SDK. The target device must have compatible runtime libraries and loader. For vendor SDKs, use their documented environment setup script and build instructions.

8. Example Makefile pattern

This minimal pattern illustrates how a Makefile can select a cross-compiler. It is not a replacement for the Makefile in your actual project.

```
CROSS_COMPILE ?= arm-none-eabi-
CC := $(CROSS_COMPILE)gcc
CFLAGS += -mcpu=cortex-m4 -mthumb -Wall -Wextra
OBJ := main.o driver.o

firmware.elf: $(OBJ)
    $(CC) $(CFLAGS) $(OBJ) -T linker_script.ld -o $@
```

```
%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

clean:
    rm -f $(OBJ) firmware.elf
```

Makefile symbols: $@ means the target filename; $< means the first prerequisite; $(OBJ) expands to the object list. The recipe lines must begin with a TAB in a traditional Makefile. This example omits header dependency generation and post-build HEX/BIN conversion.

9. Important related concepts

- **Native build vs cross-build:** A native compiler typically emits code for the host platform; a cross-compiler emits code for another target. Always confirm the actual target and flags.
- **CPU flags:** Options such as -mcpu, -march, -mthumb, and floating-point ABI options must match the target. Wrong choices may cause build errors or runtime faults.
- **Sysroot:** For embedded Linux, a sysroot provides target headers and libraries used while compiling/linking. It must match the target OS image and ABI.
- **Linker script:** Bare-metal firmware often needs a linker script describing Flash/RAM addresses, sections, vector table, stack and other memory layout details.
- **Startup code and runtime:** Startup code may initialize memory, set the vector table, and call main. C library/runtime choices affect initialization and available functions.
- **ABI and libraries:** Object files and libraries must be compatible in architecture, calling convention, data layout, and runtime assumptions.
- **Build configuration:** Board revision, RTOS, feature macros, SDK version, optimization level, and generated headers can change the resulting firmware.
- **Debug symbols and optimization:** Debug builds often keep symbols and use lower optimization; release builds may optimize for speed/size. Optimized code can be harder to step through.
- **Reproducible builds:** Pin toolchain and dependencies, record build settings, and keep artifact/version information so releases can be traced.
- **Build is not runtime validation:** A successful compile only means the configured build steps succeeded. It does not prove the firmware boots or behaves correctly on hardware.

10. Troubleshooting cross-compilation

Symptom	Likely cause	Checks / next step
command not found	Toolchain missing or PATH/environment not set	Check SDK setup, PATH, compiler filename and installed toolchain.
wrong format / architecture	Host object/library mixed with target object/library	Inspect file types and use a consistent target toolchain.
cannot find -lxxx	Target library missing or search path/sysroot wrong	Check SDK packages, -L paths, sysroot and library naming.
undefined reference	Missing implementation/library or link order issue	Check object list, library list/order, C/C++ linkage and feature configuration.
wrong instruction / FPU error	CPU or floating-point flags mismatch	Match flags to the MCU/SoC and ABI used by all linked objects.
firmware too large	Code/data exceeds Flash/RAM or sections are misplaced	Review size output, map file, linker script, optimization and feature flags.
build works but target fails	Image, ABI, runtime, startup, memory layout or hardware mismatch	Verify exact board/image, SDK, linker script, flashing process and target logs.

11. Safe checklist before building or flashing

16. Confirm the target board/CPU and whether it is bare-metal or embedded Linux.
17. Confirm the correct SDK, compiler version, ABI and environment setup.
18. Inspect the effective compile/link commands with verbose or dry-run output when uncertain.
19. Do not mix host-built libraries with target-built libraries.
20. Confirm the linker script, sysroot and startup/runtime files belong to the selected target.

21. Review the output filename, board configuration and flash target before programming hardware.
22. After building, inspect the expected artifacts and memory-size reports; then run target-level tests.

12. Short revision questions

23. What is the difference between host and target?
24. Why does an embedded MCU project often use a cross-compiler?
25. What is the difference between arm-none-eabi-gcc and aarch64-linux-gnu-gcc in typical use?
26. Why must a Linux cross-build use compatible target libraries and often a sysroot?
27. What does the linker script control in a bare-metal project?
28. What does make -n show, and what does it not guarantee?
29. Why does a successful build not guarantee the board will boot?

Reference links

- GNU Make manual: https://www.gnu.org/software/make/manual/
- GCC documentation: https://gcc.gnu.org/onlinedocs/
- GNU Binutils documentation: https://sourceware.org/binutils/docs/

Firmware Flashing, Boot Process, Root Filesystem and Image Formats

This section expands the build guide into the next part of the embedded lifecycle: how built files reach hardware, how a device starts, how Linux root filesystems work, and how archive/compression formats such as .tar, .gz, .tar.gz, .bin, .img, and .wic are used. Exact steps depend on the board, boot chain, storage type, vendor SDK and recovery design.

1. Big picture: build → package → flash → boot → verify

1. Build Compile and link source code → ELF/executable
↓
2. Convert/package Generate .bin/.hex, kernel image, DTB, rootfs image, update bundle or archive
↓
3. Transfer to target USB, UART, Ethernet, SD card, JTAG/SWD, programmer, or recovery interface
↓
4. Program storage Write bytes to Flash/eMMC/NAND/NOR/SD or another configured storage area
↓
5. Boot chain Boot ROM → bootloader → kernel (Linux) or application startup (bare metal)
↓
6. Runtime startup Initialize hardware; for Linux, mount rootfs and start init/services
↓
7. Verify Logs, version, health checks, tests, watchdog and rollback/recovery where supported

2. What does flashing mean?

Flashing means programming firmware or an image into non-volatile storage so the device can use it after power is removed. The storage may be internal MCU Flash, external NOR/NAND Flash, eMMC, or a removable SD card. In some workflows, a tool writes an image to RAM temporarily for testing instead of permanently programming storage; this is often called loading or booting an image, not flashing it permanently.

- **Firmware:** Software that controls a device. The word can mean a small MCU application, a bootloader, an RTOS image, or a larger Linux software image depending on context.
- **Flashing tool:** A program or hardware tool that communicates with the target and writes data to the intended memory region.
- **Programmer/debug probe:** Hardware such as ST-LINK, J-Link, CMSIS-DAP or a vendor probe used over SWD/JTAG; some tools also support recovery or mass erase.
- **Bootloader:** Small startup program that can initialize hardware, load the next image, verify it, and sometimes support updates or recovery.
- **Image layout:** The map that says which file goes to which storage address/partition. The same .bin file written at the wrong offset can prevent boot.

3. How the hardware receives a .bin file

A .bin file is usually a raw sequence of bytes. Unlike Intel HEX or ELF, a raw binary normally does not carry its own load address, section names, or debug symbols. The flashing tool or update manifest must know where those bytes belong.

Typical programmer flow

30. Connect the board through the correct interface (for example SWD/JTAG, USB boot mode, UART boot ROM, Ethernet recovery, or an SD card).
31. The tool identifies or connects to the target and may halt the CPU or enter a ROM/bootloader programming mode.
32. The tool erases the required Flash sectors/blocks, if needed and permitted by the target.
33. The tool writes the image in pages/blocks, often checking alignment and memory boundaries.
34. The tool may verify the written bytes by reading them back or checking a checksum/CRC.

35. The target is reset or power-cycled, and the boot chain reads the programmed image.

Background detail: Flash cannot always change arbitrary bits in place. NOR Flash commonly erases sectors before programming; NAND has page/block rules and bad-block/ECC considerations; eMMC uses block-addressed storage and partition layouts. The exact erase/write behavior is handled by the programmer, ROM loader, bootloader, or storage driver.

4. Common flashing and inspection tools

Tool / interface	Common purpose	Caution / note
OpenOCD	Debug/program supported MCUs and SoCs over JTAG/SWD with compatible probe/configuration	Target config and board wiring must match.
ST-LINK tools	Program/debug supported STM32 devices	Use a compatible ST-LINK and correct target settings.
J-Link / J-Flash	Debug/program supported targets	Licensing, device support and probe model may matter.
Vendor ROM bootloader tool	Program a device using its built-in boot ROM protocol	Requires correct boot-mode pins/procedure and supported interface.
dfu-util / USB DFU	Transfer images to targets exposing compatible USB DFU	DFU mode and image layout are device-specific.
U-Boot	Load/boot images, update storage, or provide recovery on supported embedded Linux systems	Commands and environment vary; wrong write offsets can damage bootability.
fastboot	Transfer/flash partitions on devices that implement compatible fastboot	Partition names and supported operations are device-specific.
uuu / vendor recovery utilities	Program supported SoCs through vendor-defined recovery modes	Follow the exact board/vendor procedure.
esptool.py	Flash supported Espressif ESP devices	Chip, offset and image format must match.
bmaptool / dd / balenaEtcher or similar	Write disk images to SD/eMMC media, depending on tool and image	Destination device selection is critical; a wrong disk can be overwritten.
minicom / PuTTY / screen	View serial console boot logs	Serial settings and voltage levels must be correct.

5. Commands for safe inspection and common workflows

These commands are examples for Linux host environments. Device names, addresses, programmer commands and write offsets are board-specific. Do not run destructive write/erase commands until the target device and image layout have been verified.

- **Inspect file type:** file firmware.bin — helps identify whether a file is raw data, an ELF, an archive, or another known format.
- **Inspect file size:** ls -lh firmware.bin — shows human-readable size; useful for checking whether the artifact is plausible.
- **Inspect ELF sections:** arm-none-eabi-objdump -h firmware.elf — shows section names and sizes before conversion/programming.
- **Inspect memory use:** arm-none-eabi-size firmware.elf — summarizes code/data/BSS sizes.
- **Inspect Intel HEX text:** head firmware.hex — HEX is text-encoded records; do not treat it like a raw binary.
- **Checksum:** sha256sum firmware.bin — records a cryptographic hash to compare transfer integrity; a hash alone does not prove authenticity.
- **Archive listing:** tar -tf bundle.tar — lists archive contents without extracting them.
- **Compressed file test:** gzip -t bundle.tar.gz — tests gzip stream integrity; it does not validate firmware correctness.

6. Bare-metal MCU flashing: illustrative OpenOCD example

This is a general illustration, not a universal command. The interface/target configuration filenames and reset behavior must match your probe and board.

```
openocd -f interface/stlink.cfg -f target/stm32f4x.cfg
```

OpenOCD commonly starts a server and exposes programming/debug interfaces. In a separate supported client session, users may issue commands to program, verify and reset a target. Exact commands depend on the OpenOCD version and configuration; use the board's documented procedure rather than guessing target files or memory addresses.

A safe conceptual sequence is: connect → identify target → program the intended image at its documented address → verify → reset → inspect UART logs. For production devices, consider image signing, secure boot, recovery mode and rollback protections.

7. Embedded Linux boot flow

1. Boot ROM Immutable chip code starts after reset and selects a boot source.
↓
2. First-stage loader / SPL (if used) Minimal initialization; may set up DRAM and load the next stage.
↓
3. Bootloader (often U-Boot) Chooses a boot configuration and loads kernel, device tree and optional initramfs.
↓
4. Linux kernel Initializes CPU, memory, drivers and devices; reads kernel command line.
↓
5. Root filesystem Kernel mounts the configured rootfs, such as an ext4 partition or an initramfs/cpio image.
↓
6. init / PID 1 Starts system services and applications; examples include systemd or BusyBox init.

A Linux boot image may consist of multiple separate artifacts. For example: bootloader files, kernel Image/zImage/uImage, a device tree blob (.dtb), an initramfs, and a persistent root filesystem image. Some platforms combine several pieces into one vendor-specific container.

8. What is a root filesystem (rootfs)?

The root filesystem is the directory tree mounted at / by Linux. It contains the files needed for user space to operate: programs, shared libraries, configuration, device/service setup, and system tools. It is not the same thing as the Linux kernel. The kernel manages hardware and provides system calls; user-space programs and services usually live in the rootfs.

- **/bin, /sbin, /usr/bin:** Executable programs and system utilities; exact layout depends on the distribution and usr-merge setup.
- **/lib, /usr/lib:** Shared libraries and runtime components required by applications and system services.
- **/etc:** Configuration files such as network, service and application settings.
- **/dev:** Device nodes, commonly populated/managed by devtmpfs and device-management services.
- **/proc and /sys:** Virtual filesystems exposing kernel/process and device information; not ordinary stored files.
- **/var:** Variable data such as logs, caches, databases or runtime state.
- **/home, /root:** User data and the root user's home directory, where applicable.
- **/init or systemd/BusyBox init:** The first user-space process (PID 1) that starts essential services.

9. Why, when and how rootfs is used

Use a rootfs when the target runs Linux and needs user-space tools, libraries, configuration and services. Bare-metal firmware usually does not have a Linux rootfs; it may instead use a small filesystem, EEPROM/Flash data area, or no filesystem at all.

- **Build time:** A build system such as Buildroot, Yocto/OpenEmbedded or a vendor SDK assembles the rootfs from selected packages, configuration and libraries.
- **Packaging:** The rootfs may be stored as a directory tree, tar archive, ext4 image, SquashFS image, UBIFS/UBI image, initramfs/cpio archive, or a larger disk image.
- **Boot time:** The kernel mounts the configured root filesystem, or uses an initramfs to perform early setup and mount the persistent rootfs.
- **Runtime:** Init starts services; applications use libraries and configuration from the rootfs. Persistent data may be on separate partitions.

- **Updates:** A product may update the rootfs as a whole, update selected files/packages, or use A/B partitions with rollback. The strategy is product-specific.

10. File formats and extensions you should recognize

Extension / format	What it is	Why / when used	Important distinction
.elf	Executable and Linkable Format	Linked program with sections and often symbols/debug data	May be used by debuggers/programmers; not always the production flash format.
.bin	Raw bytes	Simple firmware payload or image	Usually lacks address/section metadata; programmer must know destination offset.
.hex / Intel HEX	Text records encoding bytes and addresses	Common MCU programming format	Not the same as a raw .bin; records can carry address information.
.map	Linker map text report	Debug symbol placement and memory usage	Useful for investigating Flash/RAM overflow.
.img	Generic image filename	Could be disk, partition, firmware or vendor image	Extension alone does not define the layout; inspect documentation and file type.
.wic	Yocto/OpenEmbedded disk image format	Can describe a partitioned image for an SD/eMMC device	Specific to a build workflow; may include partition table and multiple partitions.
.dtb	Device Tree Blob	Describes hardware to the Linux kernel on many platforms	Must match board/kernel and boot configuration.
Image / zImage / uImage	Linux kernel image variants/containers	Bootloader loads the kernel	Names and container requirements depend on architecture and bootloader.
.cpio / initramfs	Archive used as an initial RAM filesystem	Early userspace before mounting persistent rootfs	May be compressed; often bundled with or passed to the kernel.
.ext4	Linux filesystem image/partition format	Persistent root filesystem or data partition	Usually mounted as a filesystem; not a generic archive.
.squashfs	Compressed, usually read-only filesystem image	Space-efficient embedded Linux rootfs	Not directly equivalent to a .tar.gz archive.
.ubifs / .ubi	Filesystem/volume formats for raw NAND workflows	Store files/volumes on supported NAND systems	Must match flash geometry and storage stack.
.tar	Archive containing multiple files and paths	Bundle source, rootfs tree, logs or update payloads	Usually not compressed by itself; extracting it creates files, not a programmed disk layout.
.gz	One gzip-compressed stream, commonly one file	Compress a file or stream	gzip alone does not bundle a directory tree; it does not preserve a full multi-file archive by itself.
.tar.gz / .tgz	A tar archive compressed with gzip	Package many files in a smaller download	First gzip decompresses the stream, then tar extracts the archive contents.
.xz / .tar.xz	XZ compression / tar archive compressed with XZ	Often used for source releases and large packages	Compression algorithm differs from gzip; tools must support it.
.zst / .tar.zst	Zstandard-compressed stream / tar archive	Fast compression/decompression in supported workflows	Requires zstd support.
.zip	Archive with built-in	Share files across operating	Not the same as tar.gz;

	compression support	systems	extraction tools and metadata behavior differ.
.swu	Common SWUpdate update bundle extension	Packaged update payload for products using SWUpdate	Format and contents depend on update configuration.
.raucb	RAUC bundle extension	Signed/structured update bundle for RAUC-based systems	Product configuration and keys determine what is accepted.
.deb / .rpm	Linux package formats	Install/update applications and libraries on supported systems	Packages are not the same as a full disk/rootfs image.
.ko	Linux kernel module	Loadable driver or kernel extension	Must match kernel version/configuration/ABI.
.dtbo	Device Tree overlay blob	Apply hardware-description changes where supported	Bootloader/kernel configuration must support overlays.
.sig / .signed	Often indicates a signature or signed artifact	Verify image authenticity where supported	Extension alone does not prove signature validity or trusted signing.

11. What exactly do tar, gz and tar.gz do?

- **tar:** Collects files and directory paths into one archive. Traditional tar does not compress the content on its own.
- **gzip / .gz:** Compresses one input stream/file. For example, gzip notes.txt commonly produces notes.txt.gz.
- **tar.gz / .tgz:** First tar bundles many files, then gzip compresses the archive. Extraction reverses the steps.

```
tar -tf package.tar.gz                # list files without extracting
tar -tzf package.tar.gz               # list files in a gzip-compressed tar archive
tar -xzf package.tar.gz -C /tmp/test  # extract into a chosen directory
gzip -t package.tar.gz                # test gzip stream integrity
sha256sum package.tar.gz              # calculate a file hash
```

Safety: list archive contents before extracting unknown packages. Extract into a temporary directory and check paths/permissions first. A valid archive or checksum does not prove the payload is trusted; use trusted sources and verify signatures when available.

12. Firmware image vs rootfs vs full disk image

Artifact	Contains	Typical use
MCU firmware .bin	Machine code and data for a microcontroller, usually linked for a known memory map	Program into the MCU's Flash at the documented address.
Linux kernel image	Linux kernel code and data	Loaded by bootloader; requires compatible boot configuration and often a DTB.
Rootfs image	Linux user-space files in a filesystem or initial userspace archive	Mounted or unpacked during boot depending on format.
Full disk image	May include partition table, bootloader areas, kernel, rootfs and data partitions	Write to SD/eMMC media using the vendor's documented method.
Update bundle	One or more payloads plus metadata, scripts and possibly signatures	Product updater validates and installs selected components.
Source archive	Source files, build scripts and documentation	Extract, configure, compile; not directly flashable.

13. How to identify an unknown file safely

36. Do not assume the meaning from the filename extension alone; .img and .bin are especially generic.
37. Use file filename and ls -lh filename to inspect the detected type and size.
38. Use archive listing commands such as tar -tf or unzip -l before extracting an archive.
39. For ELF files, use the correct readelf/objdump/size toolchain utilities to inspect architecture, sections and memory use.

40. Read the SDK/board release notes for the required flash offset, partition layout, boot mode and image format.
41. Check hashes and, where available, verify digital signatures using the product's documented trust process.
42. Never write an unknown image to a production device or disk; use a recoverable development target and follow the vendor procedure.

14. Security, recovery and production update concepts

- **Secure boot:** The device verifies boot components against trusted keys or hashes before executing them, when the platform supports it.
- **Image signing:** A signature helps verify who authorized an image and whether it changed; use the product's documented verification process.
- **Checksums vs signatures:** CRC/hash can detect accidental corruption; a cryptographic signature can authenticate origin when verified against a trusted key.
- **A/B updates:** Two bootable slots can allow the system to install an update into the inactive slot and roll back if health checks fail.
- **Recovery mode:** A documented path to restore software if normal boot fails, such as ROM bootloader, recovery partition, USB recovery or external programmer.
- **Watchdog and health checks:** Can help detect failed startup or hung software and trigger reset/rollback, depending on design.
- **Versioning and compatibility:** Track bootloader, kernel, DTB, rootfs, firmware and configuration versions; components may need compatible versions.
- **Power-loss safety:** Update design should consider interrupted writes, atomic switching, verification and rollback; do not assume all devices are safe against power loss.

15. Common failure symptoms

Symptom	Possible causes	Useful investigation
Programmer cannot connect	Wrong cable/interface, target power, reset state, boot mode, probe config	Check board guide, target voltage, wiring and tool logs.
Write/verify failure	Wrong target, protected region, bad connection, incorrect address or unsupported memory	Stop repeated attempts; verify device, image and documented address.
No boot after flashing	Wrong image/offset, missing boot component, invalid vector table, incompatible image, corrupted storage	Use serial logs, recovery procedure, known-good image and memory layout.
Linux kernel starts but cannot mount rootfs	Wrong root= argument, missing driver/filesystem support, incorrect partition/UUID, rootfs mismatch	Inspect kernel logs and bootargs; verify partition layout and filesystem support.
Kernel panic: unable to mount root fs	Kernel cannot find/read configured rootfs	Check storage driver, device tree, root= parameter, initramfs and partition names.
Rootfs boots but services fail	Missing libraries/configuration, permissions, incorrect init, read-only or full storage	Inspect serial/system logs, service status, filesystem mounts and dependencies.
Update fails or rollback occurs	Signature/version mismatch, insufficient space, interrupted update, health-check failure	Check updater logs, signing keys, version compatibility and recovery instructions.

16. Practical learning order for embedded engineers

43. Understand source code, compiler, assembler, linker, ELF, BIN and HEX.
44. Learn the project's Makefile and identify the exact build, clean, package and flash targets.
45. Identify the board, CPU, toolchain, memory map, linker script and startup code.
46. Learn how to inspect build output with size, objdump, readelf and map files.
47. Learn the board's official flashing/recovery procedure and how to capture serial boot logs.
48. For embedded Linux, understand boot ROM, bootloader, kernel, device tree, rootfs, init and services.
49. Recognize common archives, compression formats, filesystem images and update bundles.
50. Learn image integrity/authenticity, secure boot, A/B updates, recovery and power-loss risks.
51. Practice only on a development board with a known recovery path before production devices.

17. Official documentation starting points

- GNU Make: https://www.gnu.org/software/make/manual/
- GCC: https://gcc.gnu.org/onlinedocs/
- GNU Binutils: https://sourceware.org/binutils/docs/
- U-Boot documentation: https://docs.u-boot.org/
- Linux kernel documentation: https://docs.kernel.org/
- Yocto Project documentation: https://docs.yoctoproject.org/
- Buildroot manual: https://buildroot.org/docs.html
- OpenOCD documentation: https://openocd.org/doc/
- GNU tar manual: https://www.gnu.org/software/tar/manual/
- gzip manual: https://www.gnu.org/software/gzip/manual/